

Mathematical Modeling of a DC Motor

1 Equation of Motion

The first-order rotational dynamics of a DC motor with viscous friction are

$$\frac{d\omega}{dt} = \frac{T(t)}{I} - \frac{C}{I}\omega, \quad (1)$$

where $\omega(t)$ is the angular velocity, $T(t)$ is the applied torque, I is the moment of inertia, and C is the viscous damping coefficient.

2 Torque Input (PWM Pulse)

The torque is modelled as a rectangular pulse of duration t_n derived from a PWM signal:

$$T(t) = \alpha P [u(t) - u(t - t_n)], \quad (2)$$

where α is a torque constant (relating PWM to torque), P is the PWM duty-cycle amplitude, and $u(t)$ is the Heaviside unit-step function.

3 Laplace-Domain Solution for ω

Taking the Laplace transform of (1) with initial condition $\omega(0) = \omega_0$:

$$s\Omega(s) - \omega_0 + \frac{C}{I}\Omega(s) = \frac{\alpha P}{I} \frac{1 - e^{-t_n s}}{s}.$$

Solving for $\Omega(s)$:

$$\Omega(s) = \frac{\omega_0}{s + \frac{C}{I}} + \frac{\alpha P}{I} \frac{1 - e^{-t_n s}}{s(s + \frac{C}{I})}. \quad (3)$$

Using partial fractions,

$$\frac{1}{s(s + \frac{C}{I})} = \frac{I}{C} \left(\frac{1}{s} - \frac{1}{s + \frac{C}{I}} \right).$$

4 Time-Domain Solution for $\omega(t)$

Inverting (3) gives

$$\boxed{\omega(t) = \omega_0 e^{-\frac{C}{I}t} + \frac{\alpha P}{C} \left[(1 - e^{-\frac{C}{I}t}) - u(t - t_n)(1 - e^{-\frac{C}{I}(t-t_n)}) \right]}. \quad (4)$$

Equivalently (factoring as in the handwritten notes with $\alpha P/I$ out front):

$$\omega(t) = \omega_0 e^{-\frac{C}{I}t} + \frac{\alpha P}{I} \left[\frac{I}{C} - \frac{I}{C} e^{-\frac{C}{I}t} - \frac{I}{C} u(t - t_n) + \frac{I}{C} e^{-\frac{C}{I}(t-t_n)} u(t - t_n) \right].$$

5 Angular Position $\theta(t)$ by Integration

Integrating $\omega(t)$ with $\theta(0) = \theta_0$:

$$\theta(t) = \theta_0 + \int_0^t \omega(\tau) d\tau.$$

Carrying out the integration term by term:

$$\boxed{\begin{aligned} \theta(t) = & \theta_0 + \omega_0 \frac{I}{C} \left(1 - e^{-\frac{C}{I}t}\right) \\ & + \frac{\alpha P}{C} \left[t + \frac{I}{C} e^{-\frac{C}{I}t} - \frac{I}{C} \right] \\ & - \frac{\alpha P}{C} u(t - t_n) \left[(t - t_n) + \frac{I}{C} e^{-\frac{C}{I}(t-t_n)} - \frac{I}{C} \right]. \end{aligned}} \quad (5)$$

Or equivalently, factoring $\alpha P/I$ out front:

$$\begin{aligned} \theta(t) = & \theta_0 + \omega_0 \frac{I}{C} \left(1 - e^{-\frac{C}{I}t}\right) \\ & + \frac{\alpha P}{I} \left[\frac{I}{C} t + \frac{I^2}{C^2} e^{-\frac{C}{I}t} - \frac{I^2}{C^2} \right] \\ & - \frac{\alpha P}{I} u(t - t_n) \left[\frac{I}{C} (t - t_n) + \frac{I^2}{C^2} e^{-\frac{C}{I}(t-t_n)} - \frac{I^2}{C^2} \right]. \end{aligned}$$

6 Hall Sensor Observation Model

The rotor position $\theta(t)$ is observed indirectly through a binary Hall sensor. For a 4-pole motor the sensor outputs 1 when the rotor angle lies inside a window of half-width k centred on each multiple of $\pi/2$:

$$y(t) = \mathbf{1}[\exists n \in \mathbb{Z} : |\theta(t) - n\frac{\pi}{2}| < k]. \quad (6)$$

Equivalently, defining the distance from the nearest window centre

$$d(t) = \min(\theta(t) \bmod \frac{\pi}{2}, \frac{\pi}{2} - \theta(t) \bmod \frac{\pi}{2}), \quad (7)$$

the hard indicator is $y(t) = \mathbf{1}[d(t) < k]$.

6.1 Hard indicator — pole-tracking implementation

Because $\theta(t)$ is monotonically increasing during motor operation, the implementation tracks a *pole counter* p that advances whenever θ exits the current window from above, rather than recomputing the modulus at every step. For each discrete sample θ_i :

1. **Advance pole:** while $\theta_i \geq p \frac{\pi}{2} + k$, set $p \leftarrow p + 1$. (Handles the case where a single time step jumps over one or more complete windows at high angular speed.)

2. **Classify:**

$$y_i = \begin{cases} 1 & \text{if } \theta_i \geq p \frac{\pi}{2} - k, \\ 0 & \text{otherwise.} \end{cases}$$

This runs in $O(N)$ total (amortised $O(1)$ per sample) and is implemented in `omega2signal( $\theta_i, p, k$ )` which returns both y_i and the updated pole p to carry forward.

Transition detection. The binary signal changes from 0 to 1 (rising edge) or 1 to 0 (falling edge) whenever $y_i \neq y_{i-1}$. A rising edge corresponds to θ entering a window from below; a falling edge to θ leaving a window from above (or jumping directly into the next window at high speed, in which case consecutive rising/falling edges may be skipped and only the net transition is recorded).

6.2 Soft (differentiable) approximation

To enable gradient-based optimisation, the hard indicator is replaced by a smooth sigmoid:

$$\hat{p}(t; \mathbf{x}, s) = \sigma(s(k - d(t))), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (8)$$

where $s > 0$ is a *sharpness* hyperparameter. As $s \rightarrow \infty$, $\hat{p} \rightarrow y$ pointwise.

7 Parameter Identification Problem

7.1 Parameters

The five scalar parameters to be identified are

$$\mathbf{x} = (a_I, c_I, \omega_0, \theta_0, k),$$

where $a_I = \alpha/I$, $c_I = C/I$, ω_0 is the initial angular velocity, θ_0 is the initial angle, and k is the Hall window half-width.

7.2 Data alignment

The raw Arduino log (timestamps in microseconds, binary Hall state) is filtered to the interval [2.26 s, 14 s] and re-zeroed so that $t = 0$ corresponds to PWM onset. The PWM pulse runs from $t = 0$ to $t = t_n = 10$ s.

8 Loss Functions Evaluated

Let $\{(t_i, y_i)\}_{i=1}^N$ be the filtered sensor samples.

8.1 Binary MSE (`score`)

$$\mathcal{L}_{\text{score}}(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad \hat{y}_i \in \{0, 1\}. \quad (9)$$

Non-differentiable; many degenerate local minima at ≈ 0.36 .

8.2 Transition-timestamp MSE (`theta_function`)

Let $\{t_j^{\text{pred}}\}$ and $\{t_j^{\text{act}}\}$ be the predicted and actual times of 0/1 transitions. With $n = \min(|\text{pred}|, |\text{act}|)$:

$$\mathcal{L}_{\text{time}}(\mathbf{x}) = \frac{1}{n} \sum_{j=1}^n (t_j^{\text{pred}} - t_j^{\text{act}})^2 + \left(\frac{|\text{pred}| - |\text{act}|}{|\text{act}|} \right)^2. \quad (10)$$

Smoother than binary MSE but still has a flat valley where total transition count matches but timing is globally shifted; best observed score ≈ 0.46 .

8.3 Transition-interval MSE (`interval_loss`)

$$\mathcal{L}_{\text{interval}}(\mathbf{x}) = \frac{1}{n-1} \sum_{j=1}^{n-1} (\Delta t_j^{\text{pred}} - \Delta t_j^{\text{act}})^2 + \left(\frac{|\text{pred}| - |\text{act}|}{|\text{act}|} \right)^2, \quad (11)$$

where $\Delta t_j = t_{j+1} - t_j$ is the inter-transition interval (a direct proxy for instantaneous angular velocity $\omega \approx \pi/(2\Delta t)$). Immune to global phase offset; best observed score ≈ 0.46 .

8.4 Binary cross-entropy with soft Hall window (`bce_loss`)

Using the soft approximation (8):

$$\mathcal{L}_{\text{BCE}}(\mathbf{x}; s) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]. \quad (12)$$

Fully differentiable; correct Bernoulli likelihood for the observation model. Dense gradient signal (every sample contributes, not just transitions).

9 Optimisation Methods Evaluated

9.1 L-BFGS-B (`gradient-based`)

Applied to $\mathcal{L}_{\text{score}}$ initially; failed because the loss is non-differentiable. After switching to $\mathcal{L}_{\text{BCE}}$, 200 random restarts with L-BFGS-B achieved score ≈ 0.45 . Fast per restart but gets trapped in local minima.

9.2 CMA-ES (`cmaes library`)

Gradient-free covariance-matrix adaptation. Population 1000, 200 generations, parameters normalised to $[0, 1]$. Applied to $\mathcal{L}_{\text{score}}$: best score ≈ 0.363 (degenerate high-speed / small- k solution); applied to $\mathcal{L}_{\text{BCE}}$ (sharpness 50): score ≈ 0.59 .

9.3 Differential Evolution (`scipy.optimize.differential_evolution`)

Stochastic population-based global search with optional L-BFGS-B polish. Applied to $\mathcal{L}_{\text{BCE}}$:

- `popsiz=15`: BCE = 1.52, score = 0.214.
- `popsiz=100`: BCE = 0.664, score = 0.405.

Revealed an *inversion*: lower BCE does not imply lower binary MSE, because the optimizer exploits the soft window to spread uncertainty ($\hat{p} \approx 0.5$) rather than making correct hard predictions.

9.4 Sharpness annealing (`forward: s = 2 → 50`)

Sequential DE stages with increasing sharpness:

s	BCE	Score
2	0.554	0.244
5	0.438	0.189
15	0.633	0.214
50	1.506	0.216

Best score achieved: **0.189** at $s = 5$.

9.5 Sharpness annealing (reverse: $s = 5 \rightarrow 1$)

Schedule $s \in \{5, 4, 3, 2, 1\}$, each stage warm-started from the previous best (popsize=15, maxiter=500):

s	BCE	Score
5	0.4918	0.2189
4	0.5033	0.1715
3	0.5295	0.1977
2	0.5440	0.2428
1	0.6046	0.3777

Best score achieved: **0.1715** at $s = 4$.

9.6 Optimal sharpness analysis

The observed optimal $s^* \approx 4$ can be understood from the sigmoid transition width. The 10%–90% width of $\sigma(s(k-d))$ around the window edge $d = k$ is $\Delta d_{90} = \frac{\ln 9}{s} \approx \frac{2.2}{s}$. At $s = 4$, $\Delta d_{90} \approx 0.55$ rad, which is comparable to the window half-width $k \approx 0.341$ rad.

- $s \gg 1/k$: nearly hard threshold — optimizer exploits the sharp boundary by placing θ exactly at the edge, giving high confidence on the wrong side for a fraction of samples; BCE penalises these heavily, so BCE increases but score worsens.
- $s \ll 1/k$: very wide soft window — the gradient is weak everywhere; the optimizer settles for $\hat{p} \approx 0.5$ (uniform uncertainty), BCE approaches $\ln 2 \approx 0.693$, and score is near-random.
- $s \approx 1/k$ (here $s \approx 3$ –5): the sigmoid width matches the physical window width, giving the tightest alignment between the soft loss landscape and the binary MSE we ultimately care about.

9.7 Adaptive sharpness: $s = 1/k$

Since k is an unknown being optimised, fixing s manually requires re-tuning whenever k changes. The natural solution is to couple

$$s(\mathbf{x}) = \frac{1}{k},$$

computed from the current parameter estimate at every function evaluation. This keeps the sigmoid width matched to the window width throughout optimisation with no additional hyperparameter.

Result (single-stage DE, popsize=15, maxiter=1000): BCE = 0.459, score = 0.230.

Compared with the manual sharpness annealing best of 0.1715, the single-stage adaptive run is slightly worse. The annealing schedule provided implicit warm-starting that is lost here; combining adaptive sharpness with warm-starting across stages is a natural next step.

9.8 3-sensor BCE optimisation (bce_loss_3sensor)

A cosine-based soft model is used for the 3-sensor case, eliminating k entirely. With electrical angle $\varphi = 2\theta$:

$$\begin{aligned}\hat{p}_A &= \sigma(s \cos \varphi), \\ \hat{p}_B &= \sigma(s \cos(\varphi - \frac{2\pi}{3})), \\ \hat{p}_C &= \sigma(s \cos(\varphi - \frac{4\pi}{3})).\end{aligned}$$

$\cos = +1$ at the window centre, -1 at the gap centre, and 0 at the window edge (50% duty cycle), so the sharpness s acts analogously to s/k in the single-sensor model. The optimised parameters reduce to $\mathbf{x} = (a_I, c_I, \omega_0, \theta_0)$ (no k), and the loss is:

$$\mathcal{L}_{3s}(\mathbf{x}; s) = -\frac{1}{N} \sum_{i=1}^N [A_i \log \hat{p}_{A,i} + (1 - A_i) \log(1 - \hat{p}_{A,i}) + B_i(\dots) + C_i(\dots)]. \quad (13)$$

Diagnosis of failure. At steady state ($\omega_{ss} \approx 586$ rad/s) with $\Delta t \approx 0.96$ ms, the 3-bit Hall state changes every ≈ 1.07 samples. Any error in the implied ω_{ss} causes rapid phase accumulation: a 1% error places the predicted state one step wrong after ≈ 100 samples, giving $\text{BCE} \gg 2.08$ (the 3-sensor random baseline). **Result:** $\text{BCE} = 4.16$, $\text{score} = 0.439$.

A *constrained* 3-sensor variant (`bce_loss_3sensor_constrained`) fixes ω_{ss} from the 3-sensor measurement and optimises only $(c_I, \omega_0, \theta_0)$, with $a_I = \omega_{ss} c_I / P$. **Result:** $\text{BCE} = 4.24$, $\text{score} = 0.444$ — marginally worse, confirming that the 4-parameter free space was not the bottleneck; the binary cosine model itself is too phase-sensitive at this speed.

9.9 Hybrid approach: 3-sensor ω_{ss} + single-sensor BCE

Since the 3-sensor measurement of ω_{ss} is accurate (no pole-skipping, 12 transitions/revolution), while the 3-sensor BCE optimisation fails due to extreme phase sensitivity, the natural hybrid is:

1. Compute ω_{ss} from 3-sensor state transitions via $\omega_{ss} = \pi / (6 \overline{\Delta t_{\text{state}}})$.
2. Estimate k from the single-sensor duty cycle in the steady-state window: $k = \bar{y}_{ss} \pi / 4$ (unaffected by pole-skipping).
3. Optimise the full 5-parameter single-sensor BCE with all parameters free; use the 3-sensor ω_{ss} only to seed the first population row via $a_{I,\text{seed}} = \omega_{ss} c_{I,\text{seed}} / P$.

Result: $\text{BCE} = 0.727$, $\text{score} = 0.476$, implied $\omega_{ss} = 1047$ rad/s. The optimizer found a spurious solution with $\omega_0 = 771$ and $c_I = 6$ — the motor reaching steady state almost instantly, with the incorrect a_I/c_I ratio.

9.10 Sharpness as a free parameter

A wrapper `_bce_with_sharpness` expands the parameter vector to $\mathbf{x} = (a_I, c_I, \omega_0, \theta_0, k, s)$ and optimises sharpness jointly. An informed seed is placed at $s_{\text{seed}} = 1/k_{\text{meas}}$.

Result: $\text{score} = 0.451$, $s^* = 6.2$, implied $\omega_{ss} = 1047$ rad/s — sharpness landed near the expected range but the speed degeneracy remained.

9.11 Transient-only optimisation

Motivation. The steady-state plateau ($t \in [3, 10]$ s, $\approx 60\%$ of all samples) constrains only the ratio $a_I/c_I = \omega_{ss}/P$. The ramp-up ($t \in [0, 3]$ s) and coast-down ($t \in [10, 14]$ s) both exhibit curvature that independently constrains $c_I = 1/\tau$, so removing the plateau forces the optimizer to fit the transient shape.

Implementation. The data array is masked to keep only $t \leq 3$ s and $t \geq t_n$. The split index p_1 is recomputed as `searchsorted( $t_{\text{sub}}, t_n$ )` so that `_theta` still applies the correct analytical formula on each segment.

Result: $\text{BCE} = 0.678$, $\text{score} = 0.451$. The recovered $\omega_{ss,\text{implied}} = 9492$ rad/s and $k = 0.13$, indicating a degenerate solution with tiny window and near-constant $\hat{p} \approx 0.5$.

Lesson. The steady-state plateau is *necessary*: it anchors the Hall transition frequency (hence a_I/c_I). Without it, the BCE loss can be nearly minimised by predicting $\hat{p} \approx 0.5$ everywhere (BCE $\approx \ln 2 = 0.693$), making the transient shape irrelevant.

9.12 Omega_{ss} soft penalty (`_bce_omega_penalty`)

Motivation. The hard constraint $a_I = \omega_{ss} c_I / P$ couples a_I and c_I and prevents the transient from independently constraining c_I . A soft penalty preserves all data and all degrees of freedom:

$$\mathcal{L}_{\text{total}}(\mathbf{x}; s, \lambda) = \mathcal{L}_{\text{BCE}}(\mathbf{x}; s) + \lambda \left(\frac{a_I P}{c_I} - \omega_{ss} \right)^2. \quad (14)$$

The parameter vector is reduced back to $\mathbf{x} = (a_I, c_I, \omega_0, \theta_0, k)$ with s fixed per stage (not free), and ω_{ss} supplied from the 3-sensor measurement.

Single-stage with free sharpness ($\lambda = 10^{-4}$)

With sharpness also free (6 parameters), the optimizer found $s^* = 0.62$ and $k = 0.002$, i.e. $\hat{p} \approx 0.5$ everywhere. BCE = 0.685, score = 0.434; ω_{ss} matched the measurement but a_I, c_I were wrong. Conclusion: the free sharpness allows the optimizer to trivially minimise BCE by making the window nearly invisible.

Sharpness annealing ($\lambda = 10^{-4}$, schedule $s \in \{2, 3, 4, 5\}$)

With sharpness fixed per stage (5-parameter optimisation) and warm-starts:

s	Total loss	Score	Implied ω_{ss} (rad/s)
2	0.699	0.468	543
3	0.721	0.466	543
4	0.753	0.467	543
5	0.754	0.244	586

At $s = 5$ the sharpness term dominates and the penalty is overcome, allowing the implied ω_{ss} to drift from 543 to 586 rad/s. Score drops to 0.244, which is explained by steady-state dominance: matching the Hall frequency over 60% of the data (steady state) alone yields score $\approx 0.6 \times 0 + 0.4 \times 0.5 = 0.20$. Recovered $\omega_0 = 947$ rad/s confirms the optimizer found a fast-ramp degenerate solution.

Tightened ω_0 bound and stronger penalty ($\lambda = 10^{-3}$)

Two additional constraints are physically motivated:

- $\omega_0 \in [0, 10]$ rad/s: the motor is always at rest before PWM is applied, eliminating fast-ramp degenerate solutions.
- $\lambda = 10^{-3}$: at 1% ω_{ss} error the penalty contributes $10^{-3} \times (5.4)^2 \approx 0.029$, non-negligible relative to a BCE of ≈ 0.75 .

Result: score = 0.468, implied $\omega_{ss} = 540.6$ rad/s (within 0.03% of measurement), $\omega_0 = 1.5$ rad/s. However, $a_I = 0.280$, $c_I = 0.803$ — both an order of magnitude below the true values. The a_I/c_I ratio is correct (hence ω_{ss} matches), but their absolute scale is not.

Root cause: a_I - c_I scale degeneracy. For a given $\omega_{ss} = a_I P / c_I$, any pair $(\lambda a_I, \lambda c_I)$ for $\lambda > 0$ produces the same asymptotic speed. Distinguishing them requires the transient shape; but the binary BCE loss over a long steady-state window provides very weak gradient signal for the transient parameters once ω_{ss} is pinned. This degeneracy is a fundamental limitation of the single Hall sensor approach under this loss formulation.

10 Analytical Parameter Estimation Pipeline

10.1 Data alignment bug and fix

A critical bug was discovered in `data.py`: `load_data()` called `t = t - t[0]` *before* filtering to $t \geq 2.26$ s. After filtering, $t[0] \approx 2.26$ rather than 0, so every subsequent computation treated the time axis as starting at the pre-PWM epoch rather than at PWM onset.

Effect on coast-down estimation. The function `estimate_c_I_coastdown` selects transitions with $t > t_n = 10$ s. With the bug, t ran from 2.26 to 14 s, so the “coast-down” window $t > 10$ captured ≈ 1.74 s of actual deceleration preceded by ≈ 1.74 s of steady-state data (physical $t \in [10, 12.26]$ s). The log-linear slope was $\hat{c}_I \approx 0.358 \text{ s}^{-1}$ (true value 2.0 s^{-1}), an error of $5.6\times$.

Effect on optimisation. All previous DE runs were fitting a model with t misaligned by 2.26 s. The optimizer compensated with $\omega_0 \sim 944 \text{ rad/s}$ and large c_I , giving a spurious “fast-ramp” solution.

Fix. A second re-zeroing step is added *after* filtering in both `load_data()` and `load_data_3sensor()`:

```
index = t >= 2.26
t = t[index]; w = w[index]
t = t - t[0]    # re-zero to PWM onset
```

After the fix: $t = 0$ corresponds to PWM onset; the coast-down correctly begins at $t = t_n = 10$ s.

10.2 Coast-down c_I estimation

During coast-down ($t > t_n$, no applied torque):

$$\omega(t) = \omega_{ss} e^{-c_I(t-t_n)},$$

so consecutive Hall transition intervals $\Delta\tau_j$ satisfy

$$\Delta\tau_j \approx \frac{\pi/4}{\omega(t_{\text{mid},j})} = \frac{\pi}{4\omega_{ss}} e^{+c_I(t_{\text{mid},j}-t_n)}.$$

Taking logarithms:

$$\ln \Delta\tau_j = \underbrace{\ln \frac{\pi}{4\omega_{ss}}}_{\text{const}} + c_I(t_{\text{mid},j} - t_n).$$

A linear regression of $\ln \Delta\tau$ vs. $(t_{\text{mid}} - t_n)$ yields the slope $\hat{c}_I$ directly.

Result (after data-alignment fix): $\hat{c}_I = 2.024 \text{ s}^{-1}$ vs. true $c_I = 2.0 \text{ s}^{-1}$ (1.2% error).

10.3 3-sensor quantization floor at high speed

The 3-sensor steady-state estimator computes $\omega_{ss} = \pi/(6\overline{\Delta t}_{\text{state}})$, where $\overline{\Delta t}_{\text{state}}$ is the mean interval between consecutive state changes. At the operating speed:

$$\overline{\Delta t}_{\text{state}} = \frac{\pi/6}{\omega_{ss}} = \frac{\pi/6}{586 \text{ rad/s}} \approx 0.893 \text{ ms} < \Delta t = 0.962 \text{ ms}.$$

Because the motor changes Hall state *faster than the sampling rate*, every measured interval is quantized to exactly Δt . The estimator therefore returns

$$\hat{\omega}_{ss}^{(3\text{-sensor})} = \frac{\pi}{6\Delta t} = \frac{\pi}{6 \times 0.962 \text{ ms}} = 544.3 \text{ rad/s},$$

regardless of the actual speed. This is a hard quantization floor, not a statistical error: the 3-sensor estimator *cannot* produce a value above $\pi/(6\Delta t)$.

Single-sensor comparison. The single-sensor transitions correspond to $\pi/4$ rad, giving intervals of $\approx \pi/(4 \times 586) = 1.34 \text{ ms} > \Delta t$. Each interval is measurable, yielding:

$$\hat{\omega}_{ss}^{(\text{single})} = 586.1 \text{ rad/s} \quad (\text{true: } 586.2 \text{ rad/s, error } 0.02\%).$$

At this operating point, the single-sensor estimate is more accurate than the 3-sensor estimate.

The quantization floor is a general condition. The 3-sensor estimator fails whenever

$$\omega_{ss} > \frac{\pi}{6\Delta t} \approx 544 \text{ rad/s} \quad (5,200 \text{ RPM at this baud rate}).$$

10.4 Analytical estimation pipeline

Given the identified issues, parameters are recovered analytically before any numerical optimization:

1. ω_{ss} : from single-sensor transitions in the steady-state window $[3, 9] \text{ s}$ via $\hat{\omega}_{ss} = \pi/(4\overline{\Delta t})$.
2. k : from the steady-state duty cycle, $\hat{k} = \bar{y}_{ss} \pi/4$.
3. c_I : from the coast-down log-linear fit (Section above).
4. a_I : derived as $\hat{a}_I = \hat{\omega}_{ss} \hat{c}_I / P$.
5. ω_0 : fixed to 0 (motor starts from rest).
6. θ_0 : estimated via a three-phase grid search.

Phase 1: 1D grid over θ_0

Since θ_0 is periodic mod $\pi/2$, a grid of 200 points over $[0, \pi/2)$ evaluates the BCE loss for each θ_0 with all other parameters fixed at their analytic estimates. **Best score: 0.302 at $\theta_0 = 0.118$.**

Phase 2: 2D Nelder-Mead over (a_I, θ_0)

A Nelder-Mead local search refines a_I and θ_0 jointly (all other parameters fixed). **Result:** $a_I = 0.764$, $\theta_0 = -0.331$, **score** = 0.194.

Phase 3: Parallel exhaustive 2D grid

Phase drift sensitivity motivates a dense search: a 1000×200 grid over $(\omega_{ss} \pm 4 \text{ rad/s}) \times [0, \pi/2)$. For each ω_{ss} point, $a_I = \omega_{ss} c_I / P$ is derived. The grid is evaluated in parallel via `multiprocessing.Pool`.

Best result:

Parameter	Recovered	True
a_I	0.764	0.755
c_I	2.024	2.000
ω_0	0.0 (fixed)	0.000
θ_0	1.084	1.000
k	0.341	0.341
ω_{ss}	586.23	586.24
Score	0.180	

10.5 Phase drift from ω_{ss} error

At $\omega_{ss} \approx 586$ rad/s, even a small relative error ϵ in ω_{ss} accumulates rapidly:

$$\Delta\theta(t) \approx \epsilon \omega_{ss} t.$$

After $t = 10$ s of steady-state rotation, a 0.017% error (≈ 0.1 rad/s) gives $\Delta\theta \approx 1$ rad, placing approximately 50% of predictions in the wrong binary state. This is why the grid search over ω_{ss} (Phase 3) is necessary even when $\hat{\omega}_{ss}$ is already accurate to 0.02%.

10.6 Lower-PWM recommendation for reliable identification

All quantization and phase-accumulation problems become tractable at lower motor speed. The 3-sensor quantization floor is $\pi/(6\Delta t) \approx 544$ rad/s. To guarantee that 3-sensor intervals are measurable:

$$\omega_{ss} < \frac{\pi}{6\Delta t} = 544 \text{ rad/s} \iff P_{\text{scale}} \lesssim 0.93.$$

For comfortable margin (3-sensor resolution $\geq 2\Delta t$):

$$\omega_{ss} < \frac{\pi}{12\Delta t} = 272 \text{ rad/s} \iff P_{\text{scale}} \approx 0.46.$$

At $\omega_{ss} = 272$ rad/s, state-change intervals are ≈ 3.86 ms $= 4\Delta t$: well-resolved by both sensors. Phase drift over 10 s at a 0.1 rad/s error is only ≈ 1 rad, the same absolute amount but occurring over a smaller fraction of the rotation cycle. Additionally, the coast-down takes longer and provides more transitions for the log-linear c_I fit.

The `generate_data.py` script now accepts a `P_SCALE` parameter (default 1.0, set to ≈ 0.46 for the low-speed calibration run) to regenerate synthetic data at any desired operating point.

11 Computational Implementation

11.1 Vectorised score and pole=0 fix

The original `score` used a Python `while` loop for pole-tracking over $N \approx 11,000$ samples, and initialised `pole=1`, missing the Hall window at $\theta \in [-k, k]$ (the $n = 0$ pole). Both issues are fixed by replacing the loop with three vectorised NumPy operations, exploiting the equivalence

$$y(t) = \mathbf{1}[d(t) < k], \quad d(t) = \min(\theta \bmod \frac{\pi}{2}, \frac{\pi}{2} - \theta \bmod \frac{\pi}{2}).$$

This is algebraically identical to pole-tracking from `pole=0` and runs at C-speed with no Python loop.

11.2 ϕ -decomposition: vectorising over θ_0

With $\omega_0 = 0$ fixed, the analytical trajectory satisfies $\theta(t; \theta_0) = \theta_0 + \phi(t)$, where ϕ is θ_0 -independent. The grid-search inner loop over N_θ candidates is replaced by a single broadcast:

```
phi = _compute_phi(a_I, c_I, t, P, P_1, t_n)    # shape (N,)
theta = theta0_arr[:, None] + phi[None, :]      # shape (N_theta, N)
theta_mod = theta % (pi/2)
d = minimum(theta_mod, pi/2 - theta_mod)
scores = mean((d < k) - sensor, axis=1) ** 2    # shape (N_theta,)
```

For each ω_{ss} candidate, ϕ is computed once and all N_θ scores are evaluated in one pass — no inner Python loop.

11.3 Hybrid process+thread parallelism

The outer ω_{ss} grid is split into n_{proc} chunks (`os.cpu_count()`) and dispatched to a fork-context `ProcessPoolExecutor`. Within each process a `ThreadPoolExecutor` runs one thread per ω_{ss} value. Under Python 3.14 no-GIL, threads execute NumPy code in true parallel and share `sensor_data/t_s` by reference, reducing IPC overhead from N_ω to n_{proc} pickle operations.

11.4 Coarse-to-fine 2D grid

Pass A (coarse): $100 \times 25 = 2,500$ evaluations over $\omega_{ss,2d} \pm 4 \text{ rad/s} \times [0, \pi/2]$; identifies the correct phase basin.

Pass B (fine): $\text{step} = 10^{-3}$ on both axes. ω_{ss} : $\pm 0.5 \text{ rad/s}$ around the coarse best (≈ 1000 pts). θ_0 : $[\theta_0^* \pm 2k]$ (≈ 682 pts). Total $\approx 682,000$ vectorised matrix evaluations.

11.5 P_SCALE consistency

`generate_data.py` exposes `P_SCALE` (default 1.0) to simulate at reduced speed. `train.py` prompts for the same value and sets $P = 1479 \times 1.05 \times P_SCALE$. Since $a_I = \omega_{ss} c_I / P$, using different P in the two scripts produces a rescaled but otherwise correct a_I (specifically, the recovered a_I equals the true value times `P_SCALE`).

12 Key Findings and Open Issues

1. **Data alignment bug (fixed).** The original `load_data()` re-zeroed t before filtering to $t \geq 2.26 \text{ s}$, causing all model evaluations to use the wrong time origin. The fix adds a second $\mathbf{t} = \mathbf{t} - \mathbf{t}[0]$ after filtering, aligning $t = 0$ with PWM onset. All prior optimisation results (spurious $\omega_0 \sim 944 \text{ rad/s}$, $c_I \sim 6$) were artefacts of this misalignment.
2. **ω_0 lower bound.** The bound was corrected from $(1, 1000)$ to $(0, 1000)$ to allow the motor-starts-from-rest scenario.
3. **BCE / score inversion.** The soft BCE loss and binary MSE have different local minima. Higher sharpness worsens score because confident wrong predictions are heavily penalised, pushing the optimizer toward uncertain ($\hat{p} \approx 0.5$) predictions.
4. **Optimal sharpness** $s^* \approx 4 \approx 1/k$. Coupling $s = 1/k$ removes the sharpness hyperparameter entirely.
5. **Best score so far: 0.167** (analytical pipeline + coarse-to-fine 2D grid at $P_{\text{scale}} = 0.46$, $\omega_{ss} = 269.6 \text{ rad/s}$; all parameters correct to $< 1\%$ after `P_SCALE` consistency fix).
6. **3-sensor quantization floor.** At $\omega_{ss} > \pi/(6\Delta t) \approx 544 \text{ rad/s}$, the 3-sensor estimator is saturated and returns the floor value. The single-sensor estimator is more accurate in this regime (transition intervals $> \Delta t$).
7. **Phase drift bottleneck.** Even a 0.017% error in ω_{ss} causes 1 rad drift over 10 s , degrading score by $\sim 50\%$. Mitigated by the parallel exhaustive grid in Phase 3.
8. **a_I - c_I scale degeneracy.** Any pair $(\lambda a_I, \lambda c_I)$ for $\lambda > 0$ gives the same $\omega_{ss} = a_I P / c_I$. The coast-down log-linear fit breaks this degeneracy analytically by estimating c_I independently of a_I .
9. **Open next steps:** (a) collect real motor data (single + 3-bit Hall state, `micros()` timestamps); (b) apply the analytical pipeline to real data and compare recovered parameters against bench measurements of $\tau = 1/c_I$ and no-load ω_{ss} .